

Euclidean GCD on Unsigned 64-Bit Integers

LeanExe library PoC

September 2026

1 Task and interface

`LeanExe.Lib.NumberTheory.gcd` computes the greatest common divisor of two `UInt64` values using Euclid’s remainder algorithm. It accepts every pair of input words. The intended result is $\text{gcd}(a, b)$, represented as a word, with $\text{gcd}(0, 0) = 0$.

LeanExe compiles the Lean function and its callers to WebAssembly (WASM). The examples run that WASM in Wasmtime.

2 Verification status

Lean checks the source types. Execution tests pass for selected inputs: WASM and native Lean results agree with an arbitrary-precision integer reference, and the native test also checks Lean’s `Nat.gcd`. Formal correctness and termination proofs for this source function, its generated WASM, and its clients remain deferred.

3 Algorithm and source

Each iteration replaces (x, y) with $(y, x \bmod y)$ while $y > 0$. The recurrence preserves the common divisors, and the nonzero second operand decreases. When it reaches zero, the first operand is the result. NIST’s algorithm entry gives this recurrence [1]. The zero-input convention extends its positive-input description.

```
namespace LeanExe.Lib.NumberTheory

def gcd (a b : UInt64) : UInt64 := Id.run do
  let mut x := a
  let mut y := b
  while y != 0 do
    let r := x % y
    x := y
    y := r
  return x

end LeanExe.Lib.NumberTheory
```

The listing comes from `Basic.lean`. The loop performs one unsigned remainder per iteration. For $(48, 18)$, the successive pairs are $(48, 18)$, $(18, 12)$, $(12, 6)$, and $(6, 0)$. This gives result 6 after three iterations.

4 Clients and execution

The fraction client calls the component and divides both inputs by the returned divisor when the denominator is positive. Its result for 48/18 is 8/3. A zero numerator yields 0/1.

The polynomial-ratio client calls Horner evaluation with overflow checks for its numerator and denominator, then calls fraction reduction. At $x = 2$, $(1 + 3x + 2x^2)/(2 + 2x) = 15/6 = 5/2$.

The commands run from the repository root after following the setup guide.

```
tools/seminum gcd 48 18
6
tools/seminum fraction 48 18
8/3
tools/seminum ratio 2 1,3,2 2,2
5/2
```

The command-line parser accepts unsigned decimal words through $2^{64} - 1$. Invalid syntax and a zero fraction denominator produce status 2 and a message on standard error. Successful commands print their result on standard output.

5 Tests

From the repository root, `node test/seminum.js` runs the comparisons described above. Cases include zero inputs, maximum words, powers of two, consecutive large Fibonacci numbers, and generated full-width pairs. Client cases check fraction reduction, polynomial ratios, and rejection of a zero denominator. CLI tests check results and invalid-input diagnostics. The cases are defined in `test/seminum.js` and the native reference driver `test/SeminumNative.lean`.

`tools/seminum inspect` lists the component declarations found in the compiler annotations, with their catalog descriptions and proof status.

References

- [1] Paul E. Black, “Euclid’s algorithm,” NIST Dictionary of Algorithms and Data Structures, 2015. <https://xlinux.nist.gov/dads/HTML/euclidalgo.html>. The recurrence is the algorithm used here. The component specifies zero inputs and a fixed-width representation in addition to that reference.